Memos Trading — Kontext Yedekleme

Tarih: 2026-03-18 | Model: claude-sonnet-4-6 | Proje: /home/ulas/PyCharmMiscProject/memos_trading

1. Proje Genel Bakış

Alan	Değer
Workspace	Rust workspace: memos_trading_core, memos_trading_wasm, memos_trading_desktop (Tauri), trading_cli
Ana Binaryler	`main_robotic` (headless), `rtc_cli` (TUI)
TUI Stack	ratatui + crossterm
DB	SQLite (rusqlite bundled)
Exchange'ler	Binance (Spot/Futures/CoinM), BIST (Yahoo Finance), Bybit, KuCoin, Coinbase
Config Dosyaları	config/rtc_config.json, config/trade_quality.json, config/robotic_profiles.json, config/evolution_state.json

2. Kritik Modüller

Dosya	Satır	Açıklama
`robot/robotic_loop.rs`	~1700	Ana trading döngüsü — tüm optimizasyonlar burada
`robot/autonomous_control.rs`	487	FSM tabanlı otonom kontrol
`robot/autonomous_trader.rs`	557	Otonom trader (sembol performans)
`robot/backtester/engine.rs`	767	Backtest motoru
`robot/ml_engine/`	—	Linear Regression + Feature Extractor (ONNX KALDIRILDI, z-score tabanlı)
`evolution/`	—	Genetik algoritma: AdaptiveBrain Q-table + PopulationManager
`robot/order_management/`	—	Binance order yönetimi, slippage tespiti
`robot/error_recovery/`	—	CircuitBreaker, FailoverManager, RecoveryStateMachine
`robot/hot_reload/`	—	Sıfır-downtime strateji güncellemesi
`robot/portfolio_manager/`	—	Dinamik pozisyon yönetimi
`robot/position_manager.rs`	—	Phase 3 extract: OpenPosition struct, update(), realized_pnl_with_commission()
`robot/signal_evaluator.rs`	—	Phase 3 extract: TradeQualityConfig, check_quality_filters(), check_trend_filter()

3. Yapılan Mimari İyileştirmeler

Phase 1 — Pozisyon Anahtarı Düzeltmesi

Orphan pozisyon fiyat kirlenmesi sorunu çözüldü.

- Composite anahtar: `"{symbol}-{Market:?}"`
- PositionId: UUID tabanlı

Phase 2 — State Store Merkezileştirme

11× ayrı `Option<Arc<RwLock<T>>>` alan → tek `Option<SharedTradingState>` alanına indirildi.

```
pub struct TradingStateInner {
    pub live_risk:            Arc<RwLock<LiveRiskMap>>,
    pub live_price:           Arc<RwLock<LivePriceData>>,
    pub live_positions:       Arc<RwLock<LivePositionMap>>,
    pub live_strategy:        Arc<RwLock<String>>,
    pub live_regime_strategy: Arc<RwLock<String>>,
    pub live_evolution:       Arc<RwLock<LiveEvolutionStatus>>,
    pub live_active_symbol:   Arc<RwLock<String>>,
    pub live_signal_counts:   Arc<RwLock<LiveSignalCounts>>,
    pub live_trade_count:     Arc<AtomicU64>,
    pub live_closed_trades:   Arc<RwLock<ClosedTradeLog>>,
    pub live_execution_costs: Arc<RwLock<CumulativeTradingCosts>>,
}
pub type SharedTradingState = Arc<TradingStateInner>;
```

Etkilenen dosyalar: `robotic_loop.rs`, `rtc_cli.rs`, `main_robotic.rs`, `robot/mod.rs`

Phase 3 — Modül Çıkarma

- `position_manager.rs`: OpenPosition, new(), update(), realized_pnl_with_commission()
- `signal_evaluator.rs`: TradeQualityConfig, TrendBias, FilterBlock, saf fonksiyonlar

Fonksiyonel / Modern Rust Örüntüleri

Öncesi	Sonrası
23× `if let Some(ref ls) = self.config.live_FIELD { ls.write()... }`	`self.config.with_live(|st| st.update_price(|p| {...}))`
9× ayrı `live_signal_counts.write()` bloğu	`self.count_signal(SignalMetric::Buy)` tek çağrı
3-dallı if/else fiyat çözümleme	`resolve_price(a, b, c)` → `[a,b,c].into_iter().find(|&p| p > 0.0)`
2× tekrarlı pozisyon kapama + TUI bloğu	`close_position_and_log()` metoduna çekildi
2× tekrarlı LivePositionData struct init	`LivePositionData::from_pos()` constructor

4. Performans Optimizasyonları

#	Optimizasyon	Etki
1	`LazyLock<[StrategyParams; 5]> STATIC_PARAM_GRID`	Sembol başına tick başına 6× StrategyParams heap alloc ortadan kalktı
2	MarketRegime enum direct match + `regime_label: &str`	enum→&str→match çift dönüşüm ortadan kalktı
3	`interval_cycle_ids: [u64; 7]` + `interval_to_idx(s: &str) -> usize`	HashMap<String, u64> yerine stack array, candle başına String alloc yok
4	Orphan loop: `Vec<(PositionId, String)>` tek geçiş collect	collect-ID + ikinci arama çift geçişi kaldırıldı
5	Interruptible sleep: 1 sn'lik while döngüsü	Tokio scheduler ile iş birliği; Ctrl+C anında yanıt
6	`SignalMetric::BlockedTrend(&'static str)` + `Cow<'static, str>`	Statik blokaj nedenleri için sıfır alloc
7	Duplicate `#[cfg(not(target_arch = "wasm32"))]` kaldırıldı	Derleme temizliği

5. RoboticLoopConfig — Env Değişkenleri (main_robotic)

Değişken	Varsayılan	Açıklama
BINANCE_API_KEY	test_key	Binance API anahtarı
BINANCE_API_SECRET	test_secret	Binance API sırrı
BINANCE_PAPER_MODE	true	Paper trading modu
TRADE_SYMBOL	BTCUSDT	İşlem sembolü
TRADE_MARKET	spot	spot / futures / coinm
AUTONOMOUS_ENABLED	false	Otonom AI trading
USE_ML_SIGNAL	false	ML sinyal kullanımı
TRADE_CAPITAL	10000	Başlangıç sermayesi
TRADE_AMOUNT	0.01	İşlem miktarı (BTC)

6. Stratejiler

MA Crossover · RSI · MACD · Bollinger Bands · Donchian Channel · FundingRateContrarian

7. Güvenlik / Enterprise Modüller

- MFA (otpauth), JWT (jsonwebtoken), RBAC, SSO/LDAP
- HSM (pkcs11), GDPR, SIEM, Prometheus metrics
- AES-GCM şifreleme

- Enterprise modüller feature-flag ile izole: `cargo build --features enterprise`

8. Build Durumu

✓ cargo build --bin rtc_cli --bin main_robotic → 0 error, 0 warning (2026-03-18)

Son düzeltme: `LivePositionData::from_pos` → `pub(crate)` yapıldı (private_interfaces uyarısı giderildi)

9. TradingStateInner Helper API

```
impl TradingStateInner {
    pub fn update_price(&self, f: impl FnOnce(&mut LivePriceData))
    pub fn update_evolution(&self, f: impl FnOnce(&mut LiveEvolutionStatus))
    pub fn update_signal_counts(&self, f: impl FnOnce(&mut LiveSignalCounts))
    pub fn update_positions(&self, f: impl FnOnce(&mut LivePositionMap))
    pub fn read_positions<T>(&self, f: impl FnOnce(&LivePositionMap) -> T) ->
Option<T>
    pub fn try_read_risk<T>(&self, f: impl FnOnce(&LiveRiskMap) -> T) ->
Option<T>
    pub fn update_costs(&self, f: impl FnOnce(&mut CumulativeTradingCosts))
    pub fn set_regime_strategy(&self, name: &str)
    pub fn set_strategy(&self, name: &str)
    pub fn get_strategy(&self) -> String
    pub fn inc_trade_count(&self)
    pub fn append_closed_trade(&self, trade: ClosedTradeData)
    pub fn remove_position(&self, key: &str) -> bool
}

impl RoboticLoopConfig {
    #[inline] pub fn with_live(&self, f: impl FnOnce(&TradingStateInner))
    #[inline] pub fn map_live<T>(&self, f: impl FnOnce(&TradingStateInner) -> T)
-> Option<T>
}

// SignalMetric enum — count_signal() dispatch
pub enum SignalMetric {
    Buy, Sell, Hold,
    BlockedRr(Cow<'static, str>),
    BlockedVolatility(Cow<'static, str>),
    BlockedTrend(&'static str),
    BlockedRiskGate(String),
    LastParams(String),
}

// Fiyat çözümleme
fn resolve_price(candle_close: f64, live_arc: f64, last_known: f64) -> f64 {
    [candle_close, live_arc, last_known].into_iter().find(|&p| p >
0.0).unwrap_or(0.0)
}
```

10. Sonraki Adımlar (Öneriler)

- Phase 4: `robotic_loop.rs` daha da küçültmek için `trade_execution_handler.rs` modülü çıkarmak
- Backtest scheduler ile canlı canary run otomasyonu
- Prometheus metrics entegrasyonu (enterprise feature)

- Multi-symbol orchestrator ile portföy düzeyi risk yönetimi